

OPERATING SYSTEMS – HANDWRITTEN EXAM NOTES

UNIT 3: Process Synchronization, Critical Section, Semaphores, Peterson's Solution, Monitors, Dining Philosophers & Deadlocks

B.Tech Mid-1 Examination Preparation | Complete Question Bank Solutions

----- SECTION A — 2 MARKS QUESTIONS -----

Question 1

Define Deadlock

Definition

A **Deadlock** is a permanent blocking situation where a set of two or more processes are blocked forever because each process is holding a resource and waiting for another resource held by another process in the same set, so that none can proceed.

The 4 Coffman Conditions (Must hold simultaneously):

1. **Mutual Exclusion:** At least one resource is held in non-shareable mode.
2. **Hold and Wait:** A process holds at least one resource while waiting to acquire additional resources.
3. **No Preemption:** Resources cannot be forcibly taken; they must be released voluntarily by the holding process.
4. **Circular Wait:** A closed chain of processes $\{P_0, P_1, \dots, P_n\}$ exists where P_0 waits for P_1 , P_1 waits for P_2 , $\dots$, and P_n waits for P_0 .

Question 2

Define Race Condition

Definition & Example

A **Race Condition** occurs when multiple processes or threads concurrently access and manipulate shared data, and the final outcome depends unpredictably on the exact order of execution.

- **Example:** Two processes P_1 and P_2 execute `count = count + 1` concurrently when `count = 5`. Both read 5 simultaneously, increment locally to 6, and write back 6. The final value is 6 instead of the correct value 7!

- **Solution:** Enforce mutual exclusion using Semaphores, Mutex Locks, or Monitors.

Question 3

What are the requirements that a solution to the Critical Section problem must satisfy?

1. **Mutual Exclusion:** If process P_i is executing in its critical section, no other processes can be executing in their critical sections.
2. **Progress:** If no process is in its critical section and some wish to enter, the selection of the next process cannot be postponed indefinitely.
3. **Bounded Waiting:** A bound must exist on the number of times other processes are allowed to enter their critical sections after a process has requested entry, preventing indefinite waiting (starvation).

Question 4

Define Semaphores

Definition & Operations

A Semaphore is an integer synchronization variable S accessed exclusively through two atomic, indivisible operations: $\text{wait}()$ (or P) and $\text{signal}()$ (or V).

```
wait(S) { while (S ≤ 0); S--; }    signal(S) { S++; }
```

Types of Semaphores:

- **Counting Semaphore:** Value ranges over an unrestricted integer domain; used to manage access to a finite pool of resources (e.g., database connections).
- **Binary Semaphore (Mutex Lock):** Value is restricted to 0 or 1; used to guarantee mutual exclusion in critical sections.

Question 5

Name the Classic Problems of Synchronization

1. **Producer–Consumer (Bounded Buffer) Problem:** Producer inserts items into a shared buffer and consumer removes them without buffer overflow or underflow.
2. **Readers–Writers Problem:** Multiple readers can read simultaneously, but a writer requires exclusive access without starving either group.
3. **Dining Philosophers Problem:** Five philosophers share five chopsticks; models circular resource allocation without deadlock or starvation.

Question 6

What is the Critical Section Problem?

A **Critical Section** is a segment of code where a process accesses shared variables, files, or tables. The **Critical Section Problem** is the challenge of designing a protocol ensuring that only one process executes its critical section at any given time.

```
do {  
    // entry section    (request permission)  
    // critical section (access shared resource)  
    // exit section     (release permission)  
    // remainder section  
} while (true);
```

Question 7

Define Busy Waiting and Spinlock

- **Busy Waiting:** A process repeatedly executes a loop testing a condition until it becomes true, without relinquishing the CPU. It wastes CPU cycles.
- **Spinlock:** A lock implemented using busy waiting. Useful on multi-core systems when the expected wait time is extremely short (less than a context switch).

Question 8

What is Process Synchronization?

Process Synchronization is the coordination of concurrent cooperating processes sharing data to maintain data consistency and prevent race conditions.

----- SECTION B — 10 MARKS QUESTIONS -----

Essay Question 1

What is Critical Section Problem? Explain with example

1. Detailed Definition

The **Critical Section** is a code block where shared memory or devices are modified. If two processes execute their critical sections concurrently, race conditions corrupt the data.

2. Step-by-Step Race Condition Walkthrough

Consider $\text{count} = \text{count} + 1$ (starting value $\text{count} = 5$):

1. P1 executes: $\text{register1} = \text{count}$ ($\text{register1} = 5$)
2. P2 executes: $\text{register2} = \text{count}$ ($\text{register2} = 5$)
3. P1 executes: $\text{register1} = \text{register1} + 1$ ($\text{register1} = 6$)
4. P2 executes: $\text{register2} = \text{register2} + 1$ ($\text{register2} = 6$)
5. P1 executes: $\text{count} = \text{register1}$ ($\text{count} = 6$)
6. P2 executes: $\text{count} = \text{register2}$ ($\text{count} = 6 \rightarrow \text{WRONG! Should be 7}$)

3. The 3 Valid Solution Requirements

1. **Mutual Exclusion:** Only one process in Critical Section at any instant.
2. **Progress:** Only processes not in remainder section participate in deciding who enters next.
3. **Bounded Waiting:** No process waits indefinitely to enter.

Essay Question 2

What is Semaphore? Explain Producer-Consumer problem using Semaphore

1. Producer-Consumer (Bounded Buffer) Problem

A producer generates data items and deposits them into a shared cyclic buffer of size n . A consumer retrieves and processes items. The solution must guarantee that the producer never produces into a full buffer and the consumer never consumes from an empty buffer.

2. Three Semaphores Used:

- **mutex** (Binary Semaphore initialized to 1): Provides mutual exclusion for buffer access.
- **empty** (Counting Semaphore initialized to n): Tracks available empty slots in the buffer.
- **full** (Counting Semaphore initialized to 0): Tracks filled slots available for consumption.

3. Solution Algorithm Code

PRODUCER PROCESS:

```
do {  
    // produce an item in next_produced  
    wait(empty);    // decrement empty slot count (blocks if buffer full)  
    wait(mutex);    // acquire exclusive lock on buffer  
    // add next_produced to buffer  
    signal(mutex);  // release lock  
    signal(full);    // increment filled slot count (wakes up consumer)  
} while (true);
```

CONSUMER PROCESS:

```

do {
    wait(full);          // decrement filled slot count (blocks if buffer empty)
    wait(mutex);         // acquire exclusive lock on buffer
    // remove an item from buffer into next_consumed
    signal(mutex);       // release lock
    signal(empty);       // increment empty slot count (wakes up producer)
    // consume the item in next_consumed
} while (true);

```

Essay Question 3

Define Process Synchronization and explain Peterson's Solution

1. Peterson's Algorithm for Two Processes (P_i and P_j)

Peterson's Algorithm is a classic software-based solution ensuring mutual exclusion between two cooperating processes sharing two variables:

```

int turn;          // indicates whose turn it is to enter
boolean flag[2];  // flag[i] = true means  $P_i$  is ready to enter

```

Algorithm Code for Process P_i :

```

do {
    flag[i] = true;          //  $P_i$  declares interest
    turn = j;                //  $P_i$  politely gives turn to  $P_j$ 
    while (flag[j] && turn == j); // busy wait while  $P_j$  is ready and has turn
    // *** CRITICAL SECTION ***
    flag[i] = false;         //  $P_i$  exits critical section
    // remainder section
} while (true);

```

2. Proof of Correctness

- **Mutual Exclusion:** P_i enters only if $\text{flag}[j] == \text{false}$ or $\text{turn} == i$. Both processes cannot have turn equal to their own ID at the exact same instant.
- **Progress:** If P_j is not ready ($\text{flag}[j] == \text{false}$), P_i enters immediately without delay.
- **Bounded Waiting:** P_i waits at most one critical section execution of P_j before entering.

Essay Question 4

What is Monitor? Explain with any example using Monitor

1. Definition & Advantage

A **Monitor** is a high-level synchronization construct (abstract data type) that encapsulates shared variables, procedures, and condition variables. **Mutual exclusion is automatically enforced by the compiler** — only one process can be active inside the monitor at any given time.

2. Condition Variables & Operations

- `x.wait()`: The calling process is suspended and placed on condition queue `$x$`.
- `x.signal()`: Resumes exactly one process waiting on condition queue `$x$`.

3. Monitor Structure Diagram

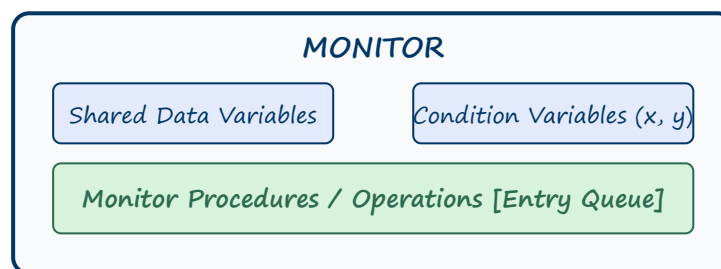


Fig 3.3: Internal Structure of a Monitor

Essay Question 5

Explain the solution for the Dining-Philosophers Problem

1. Problem Statement

Five philosophers sit around a circular table with 5 bowls of rice and 5 chopsticks. Each philosopher alternates between thinking and eating. To eat, a philosopher requires **both the left and right chopsticks**.

2. Deadlock Flaw in Naive Semaphore Approach

If each chopstick is a semaphore initialized to 1 and all 5 philosophers simultaneously execute `wait(chopstick[i])` for their left chopstick, all chopsticks are acquired and all 5 philosophers wait forever for their right chopstick → **Deadlock!**

3. Deadlock Avoidance Strategies

1. **Limitation:** Allow at most 4 philosophers to sit at the table simultaneously.
2. **Atomic Acquisition:** Pick up chopsticks only if both are available simultaneously inside a critical section.

3. **Asymmetric Solution:** Odd-numbered philosophers pick up the left chopstick first, while even-numbered philosophers pick up the right chopstick first. This completely breaks the Circular Wait condition!

----- LAST-MINUTE EXAM REVISION (UNIT 3) -----

Quick Review of Process Synchronization & Deadlocks

Concept	Key Conditions / Mechanics	Exam Significance
4 Deadlock Conditions	Mutual Exclusion, Hold & Wait, No Preemption, Circular Wait	All 4 must hold simultaneously for a deadlock to occur
Critical Section Rules	Mutual Exclusion, Progress, Bounded Waiting	Mandatory for any synchronization solution
Counting Semaphore	Unrestricted integer value	Manages resource pools with N instances
Binary Semaphore	Value 0 or 1	Equivalent to a Mutex Lock
Monitors	Automatic compiler-enforced mutual exclusion	Eliminates manual wait()/signal() lock errors